Bayesian inference on the Planck 2018 binned temperature (TT) power spectrum

Edwin Pérez
Advanced Data & Statistical Methods Laboratory

June 19, 2026

Abstract

Abstract placeholder.

1 Introduction and background

Introduction placeholder.

2 Data

We use the publicly released Planck 2018 binned temperature power spectrum, distributed as the plain-text file `COM_PowerSpect_CMB-TT-binned_R3.01.txt` in the Planck Public Data Release 3 ancillary data [1]. The file contains the CMB-only binned bandpowers, marginalised over astrophysical foregrounds and released for plotting and inspection purposes. It has five whitespace separated columns:

Column	Meaning
ℓ	effective (weighted) multipole of the bin
D_ℓ	bandpower $D_\ell = \dfrac{\ell(\ell+1)}{2\pi} C_\ell$ $[\mu\mathrm{K}^2]$
$-dD_\ell$	lower 1σ uncertainty on D_ℓ
$+dD_\ell$	upper 1σ uncertainty on D_ℓ
BestFit	Planck best-fit ΛCDM bandpower

The spectrum spans roughly $\ell \simeq 30$ to $\ell \simeq 2500$ in a few tens of bins. Throughout we work directly with D_ℓ in $\mu\mathrm{K}^2$ and treat each bin's uncertainty as independent (diagonal covariance); the consequences of this simplification are discussed in Section 4. Figure 1 shows the loaded data.

Figure 1: Planck 2018 binned TT bandpowers D_ℓ with 1σ error bars (placeholder – insert `figures/planck_tt_data.pdf` produced by the notebook).

3 Theoretical model

The theoretical TT power spectrum is computed with the Boltzmann code CAMB [3] through its Python interface (the `camb` package, installed via `pip`; no manual compilation is required). The model is parametrised by the six base ΛCDM parameters

$$\theta = \big(\Omega_b h^2,\ \Omega_c h^2,\ H_0,\ \tau,\ \ln(10^{10} A_s),\ n_s\big), \tag{1}$$

where $\Omega_b h^2$ and $\Omega_c h^2$ are the physical baryon and cold dark matter densities, H_0 the Hubble constant, τ the reionisation optical depth, A_s the amplitude and n_s the tilt of the primordial scalar power spectrum. We sample $\ln(10^{10} A_s)$ rather than A_s because it varies over a more comparable numerical range.

For a given θ, CAMB returns the lensed spectrum $D_\ell^{TT} = \ell(\ell+1)C_\ell^{TT}/2\pi$ in $\mu\mathrm{K}^2$ at integer multipoles. To compare with the binned data we evaluate the theory at the *effective* multipoles ℓ_b of each bin by linear interpolation,

$$D_b^{\mathrm{th}}(\theta) \;=\; D_\ell^{TT}(\theta)\Big|_{\ell=\ell_b}, \tag{2}$$

which approximates the Planck bin window functions (not provided in the public binned file) and is adequate because D_ℓ^{TT} varies smoothly across each bin.

Two deliberate simplifications are adopted for computational speed, both negligible at the precision of this simplified analysis: neutrinos are treated as massless ($\sum m_\nu = 0$ instead of the default 0.06 eV), and the lensing potential is computed at low accuracy. Figure 2 overlays the CAMB model evaluated at the Planck 2018 fiducial parameters on the data as a sanity check.

Figure 2: CAMB model at the Planck 2018 fiducial parameters overlaid on the binned TT data (placeholder – insert `figures/theory_vs_data.pdf` from the notebook).

4 Methodology

4.1 Likelihood

We adopt a deliberately simplified Gaussian likelihood that retains only the per-bin variance and ignores the bin-to-bin (and pixel-level) correlations of the full Planck likelihood:

$$\ln \mathcal{L}(\theta) \;=\; -\frac{1}{2}\sum_b \frac{\left(D_b^{\mathrm{data}} - D_b^{\mathrm{th}}(\theta)\right)^2}{\sigma_b^2}, \tag{3}$$

where D_b^{data} and σ_b are the bandpower and its symmetrised 1σ uncertainty from Section 2, and $D_b^{\mathrm{th}}(\theta)$ is the binned theory of Section 3. The additive normalisation constant is dropped as it does not affect the sampling. Treating the covariance as diagonal slightly *over*-states the information content relative to the official likelihood; we therefore interpret the resulting credible intervals as indicative rather than definitive, and revisit this in Section 5.

4.2 Priors

We use uniform priors on five parameters over physically reasonable ranges (Table 1) and a Gaussian prior on the optical depth,

$$\tau \sim \mathcal{N}(0.0544,\, 0.0073^2), \tag{4}$$

taken from the Planck low-ℓ EE measurement. As discussed in Section 1, TT-only data constrain A_s and τ only through the combination $A_s e^{-2\tau}$, so this prior is what breaks the degeneracy; we expect the τ posterior to essentially reproduce the prior.

The log-posterior is $\ln P(\theta \,|\, \mathrm{data}) = \ln \mathcal{L} + \ln \pi(\theta)$ up to a constant. Evaluations of the theory are wrapped so that parameter values for which the Boltzmann solver fails return $-\infty$, keeping the chains numerically robust.

Parameter	Prior	Range / (μ, σ)
$\Omega_b h^2$	uniform	[0.018, 0.026]
$\Omega_c h^2$	uniform	[0.10, 0.14]
H_0	uniform	[55, 80]
τ	Gaussian	(0.0544, 0.0073)
$\ln(10^{10} A_s)$	uniform	[2.7, 3.3]
n_s	uniform	[0.92, 1.00]

Table 1: Prior specification for the six sampled parameters.

4.3 Sampler

The posterior is explored with the affine-invariant ensemble sampler `emcee` [2]. We use $N_{\mathrm{w}} = 24$ walkers (four per dimension), initialised in a tight Gaussian ball around the Planck 2018 fiducial point, each within the prior box. The chain is run for a burn-in phase of 200 steps, which is then discarded, followed by a production phase of 1500 steps that is retained for analysis.

Because each likelihood call is dominated by a CAMB evaluation, the walker updates are parallelised with a process pool, giving a near-linear speed-up in the number of cores. The model and likelihood are packaged in a small module (`planck_model.py`); the pool uses the `spawn` start method, so each worker imports that module afresh and single-threaded. This is required on our software stack: both NumPy's OpenBLAS and CAMB use OpenMP, and forking an active OpenMP runtime deadlocks, so the `fork` start method cannot be used here. The resulting wall time scales approximately as $N_{\mathrm{w}}\,(N_{\mathrm{burn}} + N_{\mathrm{prod}})\,t_{\mathrm{CAMB}}/N_{\mathrm{cores}}$. The production chain and its log-probabilities are stored on disk so that the analysis of Section 5 can be reproduced without re-sampling.

5 Results

Results placeholder.

6 Conclusions

Conclusions placeholder.

7 Declaration of AI use

This project used the AI assistant *Claude (Anthropic)* as a coding and writing aid. Concretely, it was used to: (i) propose the project structure, (ii) draft and clean the Python code that loads the data, builds the theoretical model, defines the likelihood and runs the sampler, and (iii) help phrase parts of this report. Every code cell was read, executed and verified by the author, and all scientific choices (model, priors, simplifications) are the author's responsibility. The exact prompts that reproduce each code block are logged verbatim in Appendix A, so the degree and nature of AI assistance is fully transparent.

A Prompts log

This appendix records, step by step, a prompt that would generate the corresponding code. It doubles as the transparency record for Section 7.

Step 1 — Data loading and inspection

Prompt

"In Python, write a short, clean script (NumPy + Matplotlib only) that loads the Planck 2018 binned TT power spectrum from the local file `data/COM_PowerSpect_CMB-TT-binned_R3.01.txt`, downloading it once from the IRSA Planck PR3 mirror if it is not present. The file has five whitespace-separated columns: effective multipole ℓ, bandpower $D_\ell = \ell(\ell+1)C_\ell/2\pi$ in $\mu\mathrm{K}^2$, lower and upper 1σ errors, and the Planck best-fit. Return ℓ, D_ℓ and a symmetric error array, then plot D_ℓ versus ℓ with error bars, labelled axes and a saved `.pdf` figure. Keep comments short."

Step 2 — Theoretical model with CAMB

Prompt

"Using the `camb` Python package, write a function `theory_dl(theta)` that returns the lensed TT power spectrum $D_\ell = \ell(\ell+1)C_\ell/2\pi$ in $\mu\mathrm{K}^2$, evaluated at the effective multipoles of the Planck binned data by linear interpolation. `theta` is the six-vector $[\Omega_b h^2,\ \Omega_c h^2,\ H_0,\ \tau,\ \ln(10^{10}A_s),\ n_s]$; convert $\ln(10^{10}A_s)$ back to A_s. For speed set massless neutrinos and low lensing-potential accuracy, and compute CAMB only slightly beyond the maximum data multipole. Then plot the model at the Planck 2018 fiducial parameters on top of the data and save a `.pdf`. Keep the code short with brief comments."

Step 3 — Likelihood, priors and posterior

Prompt

"Write three NumPy functions for an MCMC fit of the six ΛCDM parameters. `log_prior(theta)` returns $-\infty$ outside a flat box (give sensible ranges for $\Omega_b h^2, \Omega_c h^2, H_0, \tau, \ln(10^{10}A_s), n_s$) and otherwise a Gaussian prior on τ centred at 0.0544 with $\sigma = 0.0073$. `log_likelihood(theta)` computes the binned theory with `theory_dl` inside a `try/except` (returning $-\infty$ if CAMB fails) and returns the diagonal Gaussian $-\frac{1}{2}\sum_b (D_b - \mathrm{model}_b)^2/\sigma_b^2$. `log_posterior(theta)` returns the sum, short-circuiting to $-\infty$ when the prior is not finite. Keep it concise."

Step 4 — MCMC sampling with emcee

Prompt

"Set up and run an `emcee` ensemble sampler over the six-parameter `log_posterior`. Use 24 walkers initialised as a small per-parameter Gaussian ball around the Planck fiducial point (assert every initial walker has finite posterior). Run a 200-step burn-in, reset, then a 1500-step production phase. Print progress every 20 steps with plain text and also append it to a log file (robust monitoring in any Jupyter). Parallelise the walker evaluations with a process `Pool` using the `spawn` start method, with the model and likelihood placed in an importable module so the workers can pickle them (`fork` is unusable here because OpenBLAS and CAMB use OpenMP). Add a `USE_POOL` toggle for a serial fallback. Save the production chain and log-probabilities to `chains/` with `np.save` and print the mean acceptance fraction. Keep the code short."

References

[1] Planck Collaboration, *Planck 2018 results. VI. Cosmological parameters*, A&A 641, A6 (2020).

[2] D. Foreman-Mackey et al., *emcee: The MCMC Hammer*, PASP 125, 306 (2013).

[3] A. Lewis, A. Challinor, A. Lasenby, *Efficient Computation of CMB anisotropies in closed FRW models*, ApJ 538, 473 (2000).